

Python Implementation

1. Overview

Python is the processing and backend layer of the AI Resume Screening System. It reads a resume, compares it with a job description using Google Gemini AI, and returns a structured screening result that the rest of the workflow can use.

This component developed in two stages:

- **Stage 1 (original contribution):** a Python screening engine consisting of PDF text extraction, Gemini-based evaluation, output validation and a JSON-based runner interface for n8n.
- **Stage 2 (later teammate work):** the engine was wrapped in a Flask API so that n8n, running in Docker, could send resumes over HTTP. Result storage and an HR dashboard were also added around it.

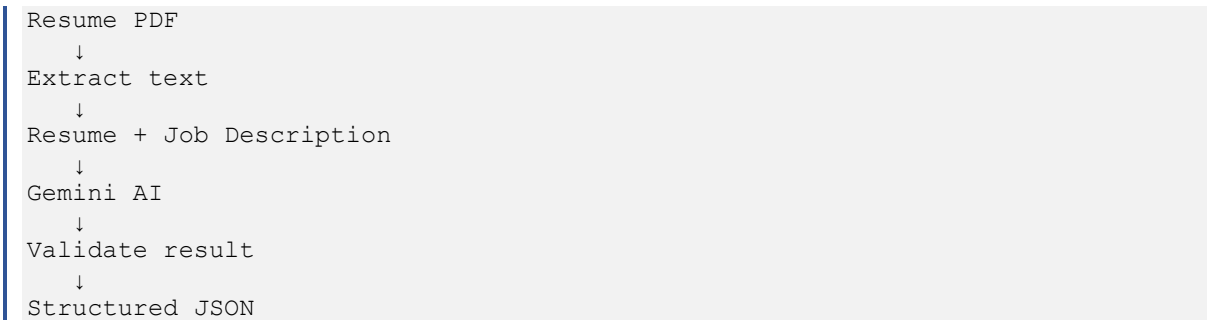
This document describes both stages and the final combined implementation. Related documents: **Ubuntu_WSL_Environment.docx** (development environment) and **Python_n8n_Integration_and_Workflow.docx** (how the components connect).

2. Initial Python Implementation

2.1 Purpose

The first version of the Python component was designed to process one uploaded resume, communicate with Gemini AI, and return a validated JSON result that n8n could consume. Keeping the AI logic in Python also kept it separate from the n8n automation workflow.

2.2 Original processing flow



2.3 Original files

File	Responsibility
score.py	Main Gemini-based evaluation logic. Receives resume text and job description, sends them to Gemini, and expects a fixed JSON result.
extract.py	Extracts text from PDF resumes using pdfplumber.

File	Responsibility
<code>n8n_runner.py</code>	Interface between n8n and the Python engine. Handles JSON input, input validation, resume file checking, PDF extraction, Gemini processing, JSON parsing, output validation, error handling and the final JSON response.

2.4 Gemini output format

The Gemini prompt was designed to return the following fields, so that n8n could process the result without interpreting free-form text:

- `candidate_name`
- `match_score`
- `key_strengths`
- `missing_skills`
- `final_verdict`
- `explanation`

2.5 Error handling

The runner was designed to handle missing files, invalid JSON, missing fields, PDF errors, Gemini errors and invalid AI responses. All errors are returned in one standard structure:

```
{
  "success": false,
  "error": "Description of the problem"
}
```

3. Changes and Enhancements Made Later

After the original engine was completed, the teammate extended the project so that the engine could be used within the n8n-based workflow. The following table summarises what changed.

Change	Type	Description
Flask API (<code>api.py</code>)	New feature; changes how Python communicates with n8n	A Flask server exposes HTTP endpoints, so n8n can call Python with an HTTP POST request.
<code>/screen</code> endpoint	Changes input format	Receives the resume as an uploaded PDF file (<code>resume</code>) and the job description as a form field (<code>job_description</code>), and returns the screening result as JSON.
Temporary resume storage	New feature	The uploaded resume is saved temporarily on disk, because the screening engine needs a real file to read.

Change	Type	Description
/result and /health endpoints	New feature	/result receives the final structured screening result; /health is available as an additional endpoint.
Result storage (data/candidates.json)	New feature	Each screening result is appended to a JSON file.
HR dashboard (app.py)	New feature	A Streamlit application reads the stored results and displays them to HR.
requirements.txt	New file	Lists the project dependencies.
Docker-to-WSL communication	Changes workflow setup	n8n (in Docker) reaches the Flask server using the WSL IP address instead of localhost.
n8n field extraction	Changes workflow	n8n extracts the result fields and passes them on for storage and display.

The original screening logic (PDF extraction, Gemini evaluation and validation) remains the core of the system. The later work built the API, storage and dashboard layers around it.

4. Final Python Implementation

4.1 Project structure

```
python-engine/
├── api.py           Flask API (later addition)
├── app.py           Streamlit HR dashboard (later addition)
├── extract.py       PDF text extraction (original)
├── score.py         Gemini evaluation (original)
├── n8n_runner.py    n8n interface (original)
├── requirements.txt Dependencies (later addition)
├── data/
│   └── candidates.json Stored screening results (later addition)
└── venv/           Python virtual environment
```

4.2 The /screen endpoint

The /screen endpoint is the entry point used by n8n. It works as follows:

1. n8n sends an HTTP POST request containing the resume PDF and the job description.
2. If no resume file is received, the API returns an error in the standard JSON format with status 400.
3. The resume is saved temporarily on disk.
4. The resume path and job description are passed to the existing screening engine.
5. The engine extracts the text, analyses it with Gemini, validates the response and produces the structured result.
6. The result is returned to n8n as JSON and stored in data/candidates.json.

A simplified part of the endpoint (added by the teammate):

```
@app.route("/screen", methods=["POST"])
def screen_resume():
```

```

job_description = request.form.get("job_description", "")
resume = request.files.get("resume")

if not resume:
    return jsonify({
        "success": False,
        "error": "No resume file received"
    }), 400

resume_path = os.path.join(
    tempfile.gettempdir(),
    resume.filename
)

resume.save(resume_path)

```

4.3 Output format

The screening result contains the same fields defined in the original Gemini prompt. An example of a stored record:

```

{
  "candidate_name": "Candidate Name",
  "match_score": 82,
  "key_strengths": ["Python", "SQL"],
  "missing_skills": ["Advanced Power BI"],
  "final_verdict": "Potential Fit"
}

```

4.4 Final flow of the Python component

```

n8n (HTTP POST: resume + job_description)
↓
Flask API /screen
↓
Temporary resume file
↓
Screening engine: extract.py + score.py
↓
Gemini AI → validated JSON result
↓
Response to n8n + storage in data/candidates.json
↓
Streamlit HR dashboard

```

5. Python Technologies and Libraries

Technology	Introduced by	Use in the project
Python	Original	Main language of the backend.
pdfplumber	Original	Extracts text from PDF resumes.
Google Gemini API	Original	Analyses the resume against the job description.
python-dotenv	Original	Loads the Gemini API key from the local <code>.env</code> file.
JSON	Original	Input parsing, Gemini output parsing, validation and result format.

Technology	Introduced by	Use in the project
Flask	Teammate (later)	Provides the HTTP API used by n8n.
Streamlit	Teammate (later)	Provides the HR dashboard that displays stored results.

6. Python–n8n Integration (Summary)

In the final version, n8n sends the resume as a binary PDF file and the job description as text to `http://<WSL-IP>:5000/screen`. Python returns the structured JSON result, and n8n extracts the result fields for the rest of the workflow. The full integration details are in **Python_n8n_Integration_and_Workflow.docx**.

7. Problems Encountered and Solutions

Problem	Solution	Stage
Gemini model returned a 404 error because the model was not available through the API route being used	Changed the model configuration to a supported model and re-tested.	Original
429 error from Gemini due to API usage limits	Retry handling was kept for temporary service failures.	Original
JSON curly braces in the prompt were treated as Python f-string expressions	Escaped the braces using <code>{{ }}</code> .	Original
Free-form Gemini responses were hard for n8n to process	Modified the prompt to require a fixed JSON structure.	Original
Invalid input or failed processing	Runner validates input and output and returns errors in a standard JSON format.	Original
Incorrect paths for candidate data and resume files caused "file not found" errors	Identified the correct project directory and used the appropriate paths.	Later
n8n passed only the filename or metadata instead of the PDF itself	Corrected the n8n HTTP Request node to send binary data (Input Data Field Name: <code>data</code>); Python reads it with <code>request.files.get("resume")</code> .	Later
n8n (Docker) could not reach the Flask server using <code>localhost:5000</code>	Used the WSL IP address (<code>http://172.19.95.137:5000</code>) instead.	Later
Result had to be structured before reaching the dashboard	n8n extracts the individual result fields before storage and display.	Later

8. Testing and Validation

8.1 Testing of the original engine

The original backend was tested with an actual resume PDF. The tests confirmed that:

- The PDF could be opened and its text extracted
- The Gemini API could be called with the resume and job description
- Gemini returned structured JSON that could be parsed
- Output fields could be validated
- Errors were returned in the standard JSON format
- `n8n_runner.py` ran successfully with a real resume PDF
- The original `score.py` continued to work

These changes were committed and pushed to GitHub.

8.2 Validation after integration

After the later changes, the team resolved the file-transfer and connection problems described in Section 7, and a resume could be sent from n8n to the Flask API, which returned the structured result. This was verified during development and has not been tested on a large scale. Stored results were checked directly in `data/candidates.json` during debugging.

9. My Contribution vs Later Team Work

My original contribution	Later modifications by teammate
PDF text extraction (<code>extract.py</code>)	Flask API (<code>api.py</code>) with <code>/screen</code> , <code>/result</code> and <code>/health</code>
Gemini evaluation logic and fixed-JSON prompt (<code>score.py</code>)	Resume upload handling and temporary file storage
n8n runner with input/output validation and standard error format (<code>n8n_runner.py</code>)	Result storage in <code>data/candidates.json</code>
Resolving Gemini model, rate-limit and f-string issues	Streamlit HR dashboard (<code>app.py</code>) and <code>requirements.txt</code>
Testing of the engine with a real resume PDF	n8n HTTP Request configuration and Docker-to-WSL connection

10. Current Limitations

- Only text-based PDFs are supported; scanned resumes need OCR, which is not implemented.
- Results are stored in a JSON file, which is not suitable for large volumes of data.

- The Flask server is started manually (`python3 api.py`) and runs only while the terminal session is active.
- The n8n connection uses a fixed WSL IP address, which may need to be updated if the network configuration changes.
- The temporary resume file is saved using the uploaded filename, and the API has no authentication.
- Scoring depends on Gemini, so results may vary between runs and are affected by API limits.

11. Future Scope

- OCR support for scanned resumes
- More advanced resume parsing (skills, education, experience, certifications, projects)
- Semantic matching using embeddings or vector databases
- Multi-factor scoring and ML-based candidate classification
- Batch processing of multiple resumes
- Replacing `candidates.json` with a database such as MySQL or PostgreSQL
- Expanding the Flask API with authentication, validation and logging
- Deployment on a Linux server or cloud platform

12. Final Outcome

Python provides the backend of the current prototype of the AI Resume Screening System. The original engine, which extracts resume text, evaluates it with Gemini and validates the output, now runs behind a Flask API. n8n can send resumes to it automatically, and the results are stored and shown to HR through a dashboard. The system is a working prototype rather than a finished product, and several improvements are planned (see Sections 10 and 11).